

Propuesta de Contrato de Arquitectura de Software

1. Introducción

El presente documento tiene como objetivo establecer las directrices técnicas y los estándares de desarrollo para el proyecto TRACLINKER. Abordará los aspectos clave relacionados con la **Arquitectura de Software**, los **Patrones Arquitectónicos** a implementar, el **Stack Tecnológico** seleccionado y los **Estándares de Codificación** que regirán el desarrollo.

Este contrato servirá como una guía fundamental para el equipo de desarrollo, asegurando la coherencia, mantenibilidad, escalabilidad y rendimiento del sistema a construir.

2. Arquitectura de Software

En esta sección se describirá la estructura de alto nivel del sistema, sus componentes principales y cómo interactúan entre sí.

2.1. Descripción General del Sistema

- **Propósito:** TracLinker permite a la empresa NEXCOM controlar inventarios, producción, incidencias de garantía y planeación de visitas técnicas, a través del entorno y las interfaces de usuario dependiendo el rol asignado por el administrador.
- **Alcance:** El software no genera facturación electrónica, Aunque tenemos la existencia de clasificación de empleados no entramos a liquidación de nómina.

2.2. Estilo Arquitectónico Principal

Se adoptará el estilo arquitectónico de **Monolítico** debido a simplicidad inicial.

2.3. Componentes Clave

- **Control de Inventarios.**
- **Gestión de Incidencias de garantía.**
- **Manejo de transformaciones.**

2.4. Consideraciones No Funcionales

Se tomarán en cuenta las siguientes características no funcionales:

- **Escalabilidad:** Capacidad para manejar un número creciente de usuarios/transacciones.
 - **Rendimiento:** Tiempos de respuesta esperados para operaciones críticas.
 - **Seguridad:** Medidas para proteger los datos y prevenir accesos no autorizados.
 - **Mantenibilidad:** Facilidad para modificar, actualizar y corregir errores en el código.
-

3. Patrones Arquitectónicos

Se detallarán los patrones de diseño y arquitectura específicos que se implementarán a lo largo del desarrollo para resolver problemas comunes y estandarizar soluciones.

MVC/MVVM: En el frontend, se seguirá el patrón MVVM para separar la lógica de negocio de la interfaz de usuario, facilitando la testabilidad y el mantenimiento.

4. Stack Tecnológico

Se listarán las tecnologías, lenguajes y frameworks que se utilizarán para la implementación del software.

4.1. Lenguajes de Programación

- **Backend:** Python v3.13.
- **Frontend:** JavaScript (ES6), Css v5.
- **Bases de Datos:** MySql 8.0.43

4.2. Frameworks y Librerías

- **Backend:** Django v5.2.5.
- **Frontend:** React v19, Tailwind v4,1.
- **Testing:** JUnit 5, Pytest.

4.3. Bases de Datos

- **Relacional:** MySql 8.0.43.

4.4. Servidores / Contenedores

- **Servidor Web/Aplicaciones:** Apache Tomcat 10
- **Contenedorización:** Docker.

4.5. Herramientas de Desarrollo y CI/CD

- **Control de Versiones:** Git (Github)
- **Integración Continua / Despliegue Continuo (CI/CD):** GitHub Actions
- **Gestión de Dependencias:** npm

5. Estándares de Codificación

Se establecerán las reglas y convenciones que todos los desarrolladores deberán seguir para garantizar la calidad, legibilidad y uniformidad del código.

5.1. Guías de Estilo de Código

Se adoptarán las siguientes guías de estilo, y cualquier desviación debe ser justificada y aprobada:

- **Python:** PEP 8.
- **JavaScript/TypeScript:** "Airbnb JavaScript Style Guide" junto con la configuración de Prettier y ESLint.

5.2. Comentarios y Documentación

- **Comentarios en Código:** Se exigirá el uso de comentarios claros y concisos para explicar lógica compleja, decisiones importantes y secciones no obvias del código. Se seguirán los estándares de docstrings para Python.
- **Documentación de APIs:** Todas las APIs deben ser documentadas utilizando **Swagger**.

5.4. Estructura de Directorios del Proyecto

Se seguirá una estructura de directorios estandarizada para todos los módulos y componentes del proyecto, facilitando la navegación y comprensión.

- src/, pages/, components/, controllers/, services/, models/, utils/, etc.

5.5. Principios de Diseño de Software

Se fomentará la aplicación de principios de diseño de software como:

- **SOLID:** Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.
- **KISS:** Keep It Simple, Stupid.
- **DRY:** Don't Repeat Yourself.

5.6. Estándares de Pruebas

- Se requerirá la escritura de pruebas unitarias y de integración para asegurar la calidad y el correcto funcionamiento de cada componente y la interacción entre ellos.
- Se establecerán objetivos de cobertura de código para las pruebas.